

Foundations of Graph neural network

[Graph neural network]

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

1.0 Content Block

where $\mathbf{\Vert}$ denotes vector concatenation, $\mathbf{0}$ is a vector of zeros, Θ is a matrix of learnable parameters, GRU is a GRU cell, and l denotes the sequence index. In a GGS-NN, the node representations are regarded as the hidden states of a GRU cell. The initial node features $x_u(0)$ are zero-padded up to the hidden state dimension of the GRU cell. The same GRU cell is used for updating representations for each node.

2.0 Content Block

Scalability and distributed training Training GNNs on large graphs presents computational challenges that differ substantially from those in other neural network architectures. Two broad problem settings arise: training on a single very large graph (such as a social network with billions of nodes), and training on collections of large individual graphs (such as molecular systems requiring higher-order interaction modeling). For the single large graph setting, sampling-based methods reduce memory requirements by operating on subgraphs rather than the full graph. Cluster-GCN partitions the graph into subgraphs using graph partitioning algorithms, training the model on each partition in sequence. GraphSAINT instead samples subgraphs stochastically during training, constructing unbiased estimators for the full graph loss. Distributed frameworks such as DistDGL extend training to multi-machine settings, managing the inter-machine communication required for neighborhood aggregation when a graph's nodes are distributed across machines. For tasks such as atomic simulations, a different bottleneck arises: GNN architectures that model higher-order interactions between triplets or quadruplets of atoms are memory-intensive at the level of individual graphs, making standard data parallelism — where each GPU processes a separate sample — insufficient when individual graphs exceed single-device memory. Graph Parallelism addresses this by distributing a single input graph across multiple GPUs, partitioning nodes and edges across devices rather than distributing samples across devices.. This approach enabled GNNs with hundreds of millions to billions of parameters to be trained on atomic systems that would otherwise exceed single-device memory, and has been applied to the development of large-scale universal interatomic potentials.

3.0 Content Block

Graph attention network The graph attention network (GAT) was introduced by Petar Veličković et al. in 2018. A graph attention network is a combination of a GNN and an attention layer. The implementation of attention layer in graphical neural networks helps provide attention or focus to the important information from the data instead of focusing on the whole data. A multi-head GAT layer can be expressed as follows:

4.0 Content Block

where $\mathbf{a}$ is a vector of learnable weights, $\cdot^T$ indicates transposition, $\mathbf{e}_{uv}$ are the edge features (if present), and LeakyReLU is a modified ReLU activation function. Attention coefficients are then normalized via softmax to make them easily comparable across different nodes. A GCN can be seen as a special case of a GAT where attention coefficients are not learnable, but fixed and equal to the edge

weights w_{uv} .

5.0 Content Block

To represent an image as a graph structure, the image is first divided into multiple patches, each of which is treated as a node in the graph. Edges are then formed by connecting each node to its nearest neighbors based on spatial or feature similarity. This graph-based representation enables the application of graph learning models to visual tasks. The relational structure helps to enhance feature extraction and improve performance on image understanding.

6.0 Content Block

Graph neural networks (GNNs) are specialized artificial neural networks that are designed for tasks whose inputs are graphs. One prominent example is molecular drug design. Each input sample is a graph representation of a molecule, where atoms form the nodes and chemical bonds between atoms form the edges. In addition to the graph representation, the input also includes known chemical properties for each of the atoms. Dataset samples may thus differ in length, reflecting the varying numbers of atoms in molecules, and the varying number of bonds between them. The task is to predict the efficacy of a given molecule for a specific medical application, such as eliminating *E. coli* bacteria. The key design element of GNNs is the use of pairwise message passing, such that graph nodes iteratively update their representations by exchanging information with their neighbors. Several GNN architectures have been proposed, which implement different flavors of message passing, started by recursive or convolutional constructive approaches. As of 2022, it is an open question whether it is possible to define GNN architectures "going beyond" message passing, or instead every GNN can be built on message passing over suitably defined graphs.

7.0 Content Block

$$\mathbf{H} = \sigma \left(\mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \mathbf{X} \Theta \right) \quad \{\displaystylestyle \mathbf{H} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{X} \mathbf{\Theta} \right) \}$$

8.0 Content Block

where K is the number of attention heads, $\mathbf{\bigvee}$ denotes vector concatenation, $\sigma(\cdot)$ is an activation function (e.g., ReLU), N_u is the set of immediate neighbor nodes of node u , including node u itself, α_{uv}^k are attention coefficients for the k -th attention head, and W^k is a matrix of trainable parameters for the k -th attention head. For the final GAT layer, the outputs from each attention head are averaged before the application of the activation function. Formally, the final GAT layer can be written as:

9.0 Content Block

$$\alpha_{uv} = \exp \left(\mathbf{\bigvee}_{z \in N_u} \text{LeakyReLU} \left(\mathbf{a}^T [W^k \mathbf{x}_u \mathbf{\bigvee} W^k \mathbf{x}_z \mathbf{\bigvee} \mathbf{e}_{uv}] \right) \right) \sum_{z \in N_u} \exp \left(\mathbf{\bigvee}_{z \in N_u} \text{LeakyReLU} \left(\mathbf{a}^T [W^k \mathbf{x}_u \mathbf{\bigvee} W^k \mathbf{x}_z \mathbf{\bigvee} \mathbf{e}_{uz}] \right) \right) \quad \{\displaystylestyle \alpha_{uv} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [W \mathbf{x}_u \mathbf{\bigvee} W \mathbf{x}_v \mathbf{\bigvee} \mathbf{e}_{uv}]))}{\sum_{z \in N_u} \exp(\text{LeakyReLU}(\mathbf{a}^T [W \mathbf{x}_u \mathbf{\bigvee} W \mathbf{x}_z \mathbf{\bigvee} \mathbf{e}_{uz}]))} \}$$